

Numerical Method Used for the Flow Around the Cube

0. Why a numerical method is needed here

As shown for the cylinder and the sphere, when the obstacle has a "rounded" shape (circle, sphere, ellipse) a closed-form formula exists because Laplace's equation can be separated in a coordinate system where that shape is a constant-coordinate surface. A cube has sharp corners, so that coordinate system does not exist and we also want to capture the real viscous physics (flow separation, wake, drag), which potential flow does not describe. That leaves no option but to solve the full Navier-Stokes equations numerically: the continuous domain is replaced by a discrete grid of points, derivatives are replaced by differences between those points, and the solution is advanced in time step by step.

1. The equations being solved

1.1 Conservation of mass (incompressibility)

$$\nabla \cdot V = 0$$

1.2 Conservation of momentum (Navier-Stokes)

$$\partial V / \partial t + (V \cdot \nabla) V = - (1/\rho) \nabla p + \nu \nabla^2 V$$

Reading each term from left to right:

- $\partial V / \partial t$: how velocity changes in time, at a fixed point in space.
- $(V \cdot \nabla) V$: the convection term .
- $-(1/\rho) \nabla p$: the force per unit mass due to the pressure gradient.
- $\nu \nabla^2 V$: viscous diffusion ($\nu = 1/\text{Re}$ is the kinematic viscosity) .

Unlike potential flow, here the $\nu \nabla^2 V$ term is present, and it is precisely what allows the flow to separate behind the cube but it is also what makes the equation lack an analytical solution for geometries like this one.

2. Spatial discretization: the grid

The domain (a $4 \times 4 \times 4$ tunnel, with the cube centered inside it) is replaced by a regular grid of $n_x \times n_y \times n_z$ points ($35 \times 35 \times 35$ in the original script), separated by a constant step:

$$\Delta x = \Delta y = \Delta z = L / (n - 1)$$

At each of those points a value of u , v , w and p is stored: these are the arrays `self.u`, `self.v`, `self.w`, `self.p` in the code, each of shape (n_x, n_y, n_z) .

Simplification used in the code: this is a "collocated" grid (all variables live at the same points). More accurate CFD solvers use "staggered" grids (MAC grids), where pressure and each velocity component are stored at slightly offset points, this avoids a well-known checkerboard pressure-oscillation problem. Here the simpler (collocated) version was used on purpose, to keep the code easier to read, at the cost of some of that extra robustness.

2.1 The cube inside the grid

No geometric cube is drawn: for every point of the grid, we mark whether it falls "inside" or "outside" the cube (a boolean mask), by comparing the distance to each axis against half the cube's side length:

$$\text{mask}(i,j,k) = |x_i - x_c| \leq L_{\text{cube}}/2 \text{ AND } |y_j - y_c| \leq L_{\text{cube}}/2 \text{ AND } |z_k - z_c| \leq L_{\text{cube}}/2$$

At every point where that mask is true, $u=v=w=0$ is enforced (the solid does not move, and the fluid cannot be "inside" it).

3. Replacing derivatives with finite differences

The core idea of any finite-difference method: a derivative is, by definition, the limit of an incremental ratio. If we do not take the limit ($h \rightarrow 0$) and instead keep a small but finite Δx , we get an approximation:

$$\begin{aligned}\partial f / \partial x &\approx (f(x+\Delta x) - f(x-\Delta x)) / (2\Delta x) && \text{(central difference, error } \sim \Delta x^2) \\ \partial^2 f / \partial x^2 &\approx (f(x+\Delta x) - 2f(x) + f(x-\Delta x)) / \Delta x^2 && \text{(1D Laplacian, error } \sim \Delta x^2)\end{aligned}$$

The full Laplacian (∇^2) is built by summing this second derivative along the 3 directions. This is exactly what the code does in the variables `lap_u`, `lap_v`, `lap_w`: for every interior point, it looks at its 6 neighbors ($\pm x, \pm y, \pm z$) and combines them this way.

3.1 The trouble with convection: central differencing vs. upwind

For the convection term $(V \cdot \nabla)V$ one could also use the central-difference formula above. The problem is that this approximation is only stable when the cell Péclet number is not too large:

$$Pe = U \cdot \Delta x / \nu \leq 2 \quad \text{(stability condition for central differencing)}$$

With the grid used here ($\Delta x \approx 0.118$) and $Re=50$ ($\nu=0.02$), $Pe \approx 6$, well above that limit. With central differencing, this produces numerical oscillations (a "checkerboard" pattern) that have nothing to do with the real physics.

The fix applied was switching to a first-order upwind scheme: instead of looking at the neighbor ahead AND the one behind at the same time, only the neighbor on the side the flow is coming FROM is used:

$$\partial f / \partial x \approx (f_i - f_{i-1}) / \Delta x \quad \text{if } u \geq 0, \quad (f_{i+1} - f_i) / \Delta x \quad \text{if } u < 0$$

This is exactly what the `_grad_upwind` function in the code does, using `np.where` to pick the formula based on the sign of the local velocity.

Cost of this fix: the upwind scheme is unconditionally stable for any Pe , but it introduces extra NUMERICAL viscosity, of order $\nu_{num} \approx U \Delta x / 2$. On this grid, $\nu_{num} \approx 0.059$, almost 3 times the physical viscosity ($\nu=0.02$ for $Re=50$). This means the Reynolds number actually being resolved is lower than the nominal one (effective $Re \approx 10$, not 50) exactly the point discussed earlier. To reduce this numerical viscosity without losing stability, the two options are refining the mesh (a smaller Δx reduces ν_{num} proportionally) or using a higher-order scheme (QUICK, second-order upwind, TVD), which has less numerical diffusion than first-order upwind.

4. Advancing in time: explicit Euler

With the spatial derivatives already approximated, the time derivative is also approximated with a forward difference:

$$\partial V / \partial t \approx (V^{n+1} - V^n) / \Delta t$$

Solving for the value at the next step ($n+1$) from everything already known at the current step (n):

$$V^{n+1} = V^n + \Delta t \cdot (\nu \nabla^2 V^n - (V^n \cdot \nabla) V^n)$$

This is called "explicit" because the right-hand side uses only values ALREADY known from step n . No system of equations needs to be solved for this part (unlike pressure, which does, see Section 5). It is the simplest part to implement, but it carries a strict stability limitation (Section 6).

5. The pressure problem: the projection method

If velocity is updated using ONLY the momentum equation (Section 4), without touching pressure, the result will almost certainly stop satisfying the incompressibility condition ($\nabla \cdot V = 0$). The projection method (or Chorin's method) fixes this in 3 steps within every time step:

Step A — Provisional velocity (predictor)

An intermediate velocity V^* is computed using diffusion + convection only, IGNORING pressure (this is exactly the formula from Section 4). This V^* generally does not satisfy $\nabla \cdot V^* = 0$.

Step B — Pressure Poisson equation

A pressure correction is sought such that, once applied, the final velocity DOES satisfy incompressibility. Starting from the correction formula (seen in Step C) and requiring $\nabla \cdot V^{n+1} = 0$, one arrives at the condition that pressure must satisfy:

$$\nabla^2 p = (\rho / \Delta t) \nabla \cdot V^* \quad (\text{Poisson equation})$$

This is again an equation from the Laplace/Poisson family (the same family that appeared in potential flow), but now with a "source" term on the right-hand side (the residual divergence of V^*), and unlike the potential-flow case, it is solved numerically, point by point, with the iterative Gauss-Seidel method: the grid is swept many times (40 iterations in the code, inside every time step), and on each pass the pressure at each point is updated as a weighted average of its 6 neighbors, corrected by that divergence:

$$p_{i,j,k} \leftarrow [\text{weighted sum of } p \text{ over the 6 neighbors} - (\Delta x^2 \Delta y^2 \Delta z^2 / \Delta t) \cdot (\nabla \cdot V^*)] / (2 \cdot \text{sum of weights})$$

Each Gauss-Seidel iteration "spreads" the pressure correction a bit further across the grid; after enough iterations, the field converges to (an approximation of) the solution of the Poisson equation.

Step C — Velocity correction

With pressure already computed, the provisional velocity is corrected by subtracting the pressure gradient (scaled by $\Delta t / \rho$):

$$V^{n+1} = V^* - (\Delta t / \rho) \nabla p$$

This V^{n+1} now does satisfy (approximately, depending on how well Gauss-Seidel converged) the incompressibility condition, and it is used as the starting point for the next time step.

In the code, this A→B→C cycle is exactly what the solve_step() function does: it first updates u,v,w with diffusion+convection (Step A), computes the divergence div, solves pressure with the Gauss-Seidel loop (Step B), and finally corrects u,v,w by subtracting the pressure gradient (Step C).

6. Why Δt cannot be arbitrary: numerical stability

Because the time scheme is explicit (Section 4), Δt cannot be chosen freely: if it is too large, the error is amplified at every step and the result "blows up" (values growing without bound, typically ending in NaN). Two limits must be respected at the same time:

6.1 Convective limit (CFL condition)

Information cannot numerically "travel" faster than one grid cell per time step:

$$\Delta t < \Delta x / U_{max}$$

6.2 Diffusive limit

Diffusion also cannot spread more than one cell per step:

$$\Delta t < \Delta x^2 / (6\nu) \quad (\text{the 6 comes from summing the 6 neighbors in 3D})$$

The code computes Δt automatically, taking the more restrictive of the two, with a safety margin (factor 0.4-0.5) so as not to sit right at the limit:

$$\Delta t = 0.4 \cdot \min(0.5 \cdot \Delta x / U_{max}, 0.5 \cdot \Delta x^2 / (6\nu))$$

7. Boundary conditions

Boundary	Condition applied	Physical meaning
Inlet ($x=0$)	$u=U_0, v=w=0$	Wind entering the tunnel uniformly
Outlet ($x=L$)	zero gradient (copies the previous value)	Flow leaves freely, nothing is forced
Side walls	$u=v=w=0$	No-slip: air does not move right at the wall
Cube surface	$u=v=w=0$	No-slip: air neither crosses nor slides on the solid
Pressure at outlet	$p=0$	Pressure reference ($p_\infty=0$), needed because Poisson only determines p up to a constant

Honest note on a simplification: the code does not impose a specific pressure boundary condition on the cube's surface (it should also be $\partial p / \partial n = 0$ there, similar to the side walls). This is an acceptable approximation for a demo, but it can introduce some extra smoothing near the cube's edges.

8. Final pressure coefficient calculation

Once the simulation reaches a (quasi) steady state (after enough time steps), C_p is computed with the same Bernoulli formula used in the analytical case (cylinder/sphere). The difference is that here p no longer comes from a closed-form formula, but from the field that the iterative solver built step by step:

$$C_p = (p - p_\infty) / (\frac{1}{2}\rho U_0^2) = 2p / U_0^2 \quad (\text{with } \rho=1, p_\infty=0)$$

A light Gaussian smoothing ($\sigma \approx 0.6-0.8$) is then applied purely to clean up grid noise for plotting.

9. Summary of the full algorithm

1. Build the 3D grid and mark which points fall inside the cube.
2. Set the initial condition: uniform wind U_0 throughout the tunnel, zero inside the cube.
3. Compute Δt automatically (convective + diffusive CFL).
4. Repeat many times (600 steps in the corrected version):
 - 4a. Compute diffusion (Laplacian, central differences) and convection (upwind) $\rightarrow$ provisional velocity V^* .
 - 4b. Re-apply the boundary conditions.
 - 4c. Compute the divergence of V^* (how much incompressibility is being violated).
 - 4d. Solve the pressure Poisson equation with 40 Gauss-Seidel iterations.
 - 4e. Correct velocity with that pressure's gradient $\rightarrow V^{n+1}$.
 - 4f. Re-apply the boundary conditions (in case any step disturbed them).
5. Once the loop finishes, compute $C_p = 2p/U_0^2$ over the whole domain.
6. Plot results (2D slices with streamlines, 2D C_p map, 3D C_p isosurfaces).

10. Known limitations of this method

- Collocated grid (not staggered): simpler to implement, somewhat less robust against pressure oscillations than a MAC grid.
- First-order upwind: stable, but adds non-negligible numerical viscosity on this mesh: the effective Reynolds number resolved is lower than the nominal one (see Section 3.1).
- No explicit pressure boundary condition on the cube surface: an acceptable approximation, but it can over-smooth near the edges.
- Gauss-Seidel with a fixed number of iterations (40): does not necessarily converge fully at every time step, especially in the early steps; it is a trade-off between accuracy and computation time.
- Explicit Euler: the simplest possible time-stepping scheme; higher-order methods (Runge-Kutta) give more accuracy for the same Δt , at the cost of more computation per step.

11. Sample results

The figures below show representative results from running this solver on a $35 \times 35 \times 35$ grid at $Re=50$ (nominal), after 600 time steps, once the flow has reached a quasi-steady state.

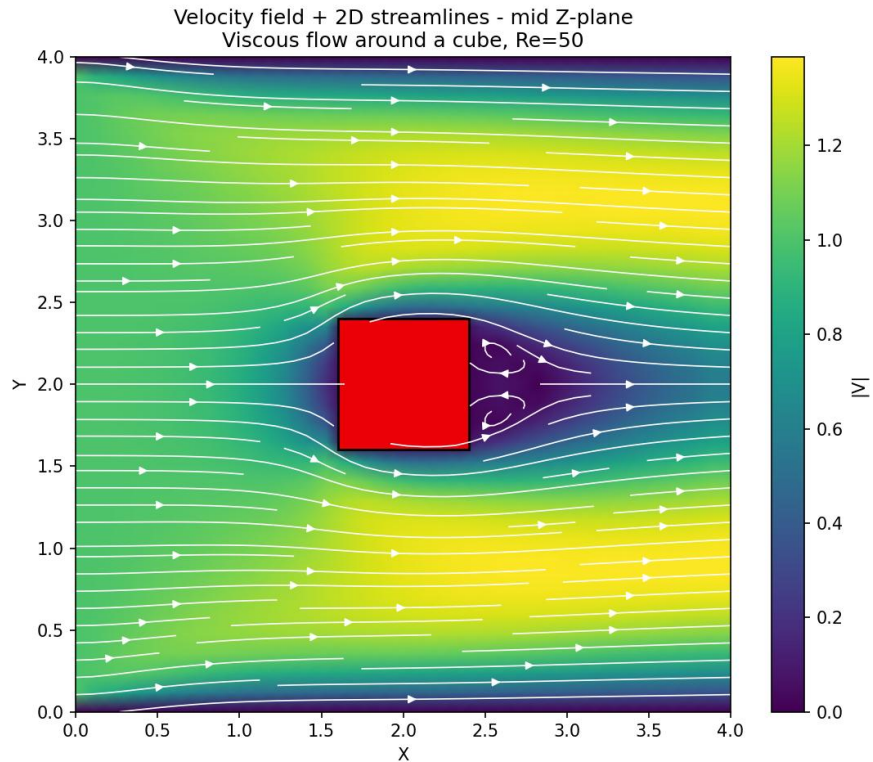


Figure 1. Velocity magnitude field and 2D streamlines on the mid Z-plane. A small recirculating wake is visible directly behind the cube: the low-speed (dark) region with two counter-rotating vortices, a signature of viscous separation that potential flow cannot reproduce.

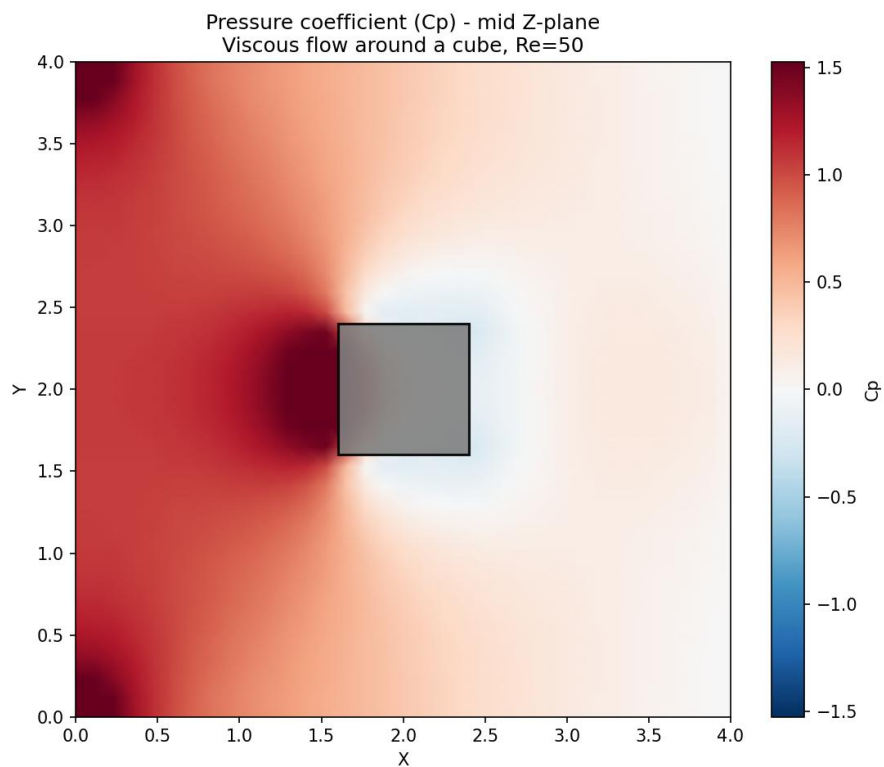


Figure 2. Pressure coefficient (C_p) on the mid Z-plane. The dark red region upstream of the cube is the stagnation zone (C_p approaching its theoretical maximum of 1); the blue region around the top/sides and wake is the low-pressure suction zone, consistent with the classical potential-flow trend but shaped by real flow separation.

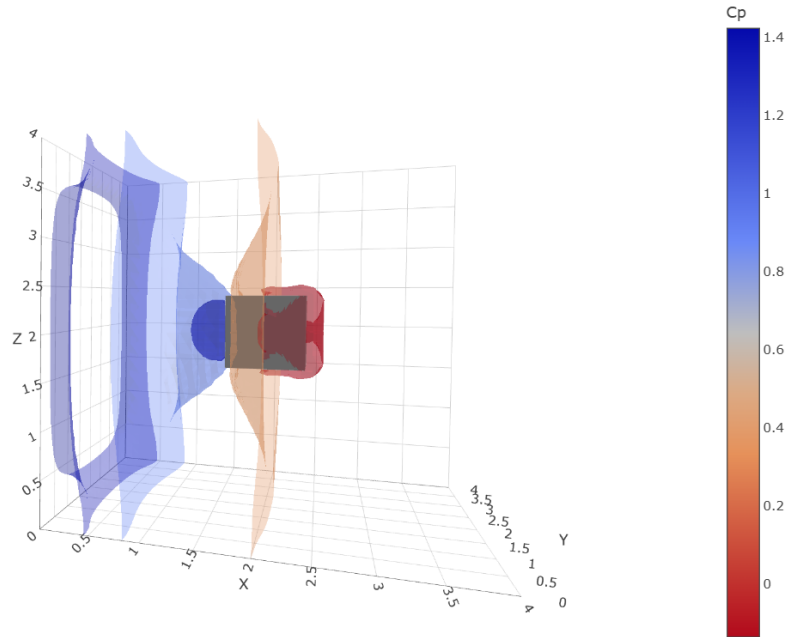


Figure 3. Three-dimensional C_p isosurfaces around the cube. Blue: high-pressure stagnation shell upstream ($C_p \approx +1.4$); Red: low-pressure suction shell wrapping the top, sides and wake ($C_p \approx -0.13$). The gray box is the cube itself..